A Complete Guide to Styled Components

1. What are Styled Components?

Styled Components is a popular **third-party package** for React. It lets you write real CSS inside your JavaScript files, so each component carries its own styles — no clashes, no mess.

Why use Styled Components?

- Styles are **scoped to one component** only — they won't leak to others.
- You write **normal CSS** syntax (no camelCase tricks needed).
- Styles live right next to the component code — easy to find and change.
- Works great with **React function components**.

How to install it:

```
# Run this command in your project folder:
npm install styled-components
```

Basic usage example:

```
import styled from 'styled-components';

// Create a styled button
const MyButton = styled.button`
background-color: #2E86AB;
color: white;
padding: 10px 20px;
border: none;
border-radius: 6px;
cursor: pointer;
`;

// Use it like a normal React component
function App() {
return Click Me;
}
```

■ **Tip:** The backtick (`) after *styled.button* is called a **tagged template literal** — it's just a special way to pass text in JavaScript.

2. Creating Flexible Components with Styled Components

Styled Components makes it easy to build **flexible**, reusable UI pieces. You can extend styles, pass in custom values, and combine components freely.

Extending / inheriting styles:

```
const BaseButton = styled.button`
padding: 10px 18px;
border-radius: 4px;
font-size: 14px;
`;

// Extend BaseButton with extra styles
const PrimaryButton = styled(BaseButton)`
background: #1A5276;
color: white;
`;

const OutlineButton = styled(BaseButton)`
background: transparent;
border: 2px solid #1A5276;
color: #1A5276;
`;
```

Attaching styles to any HTML tag:

```
const Title = styled.h1` font-size: 2rem; color: #1A5276; `;
const Card = styled.div` background: #f9f9f9; padding: 20px; `;
const Link = styled.a` color: #2E86AB; text-decoration: none; `;
```

■ **Tip:** You can use **styled.div**, **styled.p**, **styled.input**, or any HTML element — just replace the part after the dot.

3. Dynamic & Conditional Styling

One of the best features of Styled Components is that you can change styles based on **props** (data passed into the component). This means one component can look different depending on its state.

Dynamic styling using props:

```
const Button = styled.button`
background: ${props => props.primary ? '#1A5276' : '#AED6F1'};
color: ${props => props.primary ? 'white' : '#1A5276'};
padding: 10px 20px;
border: none;
border-radius: 5px;
`;

// Usage:
Primary Button
Secondary Button
```

Conditional class-like logic with css helper:

```
import styled, { css } from 'styled-components';

const Input = styled.input`
border: 2px solid #ccc;
padding: 8px;
${props => props.hasError && css`
border-color: red;
background: #fff0f0;
`}
`;

// Usage:
```

■ **Tip:** Use the **css** helper when you want to apply a *block* of CSS conditionally — it keeps the code clean and readable.

4. Pseudo Selectors, Nested Rules & Media Queries

Styled Components supports the full power of CSS — including hover effects, focus states, child selectors, and responsive breakpoints — all inside the template.

Pseudo selectors (hover, focus, active):

```
const Button = styled.button`
background: #2E86AB;
color: white;
padding: 10px 20px;

&:hover {
background: #1A5276; /* darker on hover */
}

&:focus {
outline: 3px solid #AED6F1;
}

&:active {
transform: scale(0.97);
}
`;
```

Nested rules (targeting child elements):

```
const Card = styled.div`
background: #f9f9f9;
padding: 20px;

h2 {
color: #1A5276; /* styles h2 INSIDE this card only */
margin-bottom: 8px;
}

p {
font-size: 14px;
color: #555;
}
`;
```

Media Queries (responsive design):

```
const Container = styled.div`
width: 100%;
padding: 20px;

@media (min-width: 768px) {
width: 700px; /* wider layout on tablets */
margin: 0 auto;
}

@media (min-width: 1200px) {
width: 1100px; /* even wider on desktops */
}
`;
```

■ **Tip:** The **&** symbol refers to the current component itself. It works just like the & in SCSS/SASS, so SASS users will feel right at home!

5. Reusable Components & Component Combinations

The real power of Styled Components comes when you build a **library** of small, reusable styled pieces and combine them to form bigger UI sections.

Example — building a Card from smaller pieces:

```
const Card = styled.div` background: white; border-radius: 8px;
box-shadow: 0 2px 8px rgba(0,0,0,0.1);
padding: 24px; `;
const CardTitle = styled.h3` font-size: 1.2rem; color: #1A5276; margin: 0 0 8px; `;
const CardText = styled.p` font-size: 14px; color: #555; line-height: 1.6; `;
const CardButton = styled.button` background: #2E86AB; color: white;
padding: 8px 16px; border: none;
border-radius: 4px; cursor: pointer; `;

// Combine them:
function ProductCard({ title, description }) {
return (

{title}
{description}
Learn More

);
}
```

■ Pros

- ✔ One component = one style block → easy to find and fix.
- ✔ No name collisions — styles are auto-scoped.
- ✔ Props make components very flexible without extra CSS classes.
- ✔ Moving a component to another project brings its styles too.
- ✔ Works perfectly with React's component model.

■ Cons

- ✘ Adds a third-party dependency to your project.
- ✘ Styles are hidden inside JS files — harder for pure CSS devs.
- ✘ Slight performance overhead at runtime (styles injected via JS).
- ✘ Learning curve: template literals + props logic take some getting used to.
- ✘ Large projects can lead to many small styled components everywhere.

6. Quick Reference – Common Patterns

What you want	Styled Components way
Basic styled element	`styled.div`...` / styled.button`...`

Hover / focus effect	`&:hover { ... } inside the template`
Style child elements	`h2 { ... } or & > span { ... }`
Change style via prop	`${props => props.active ? 'blue' : 'grey'}`
Conditional CSS block	`${props => props.error && css`border: red;`}`
Responsive breakpoint	`@media (min-width: 768px) { ... }`
Extend another component	`styled(BaseButton)`extra styles``

7. Code Examples

All the code examples shown in this guide (and more!) are available on GitHub. Click the link below to browse the full example project:

https://github.com/academind/react-complete-guide-course-resources/tree/main/code/07%20Styling

■ **How to use the examples:** Clone the repo → open the *07 Styling* folder → run **npm install** → run **npm start**. Each sub-folder matches a topic covered in this guide.